

# EC2 Volume Project: Comprehensive Accomplishments and Skills Demonstrated

## Project Overview

This document presents a detailed account of the EC2 Volume Project, highlighting the completed accomplishments and the technical skills demonstrated throughout each stage. The project focused on AWS infrastructure management—including EC2 instance setup, EBS volume operations, S3 bucket management, data migration, and disaster recovery strategies.

## EC2 Instance Setup

An EC2 instance named `VolumelInstance` (instance ID: `i-00000000000000000000`) was launched using the `t2.micro` type in the US East (N. Virginia) region, specifically in the `us-east-1b` availability zone. The operating system chosen was Amazon Linux 2023. SSH access to the instance was successfully established, and the instance’s public IP address was `34.229.206.11`.

|  |                 |  |  |         |  |          |  |                   |                               |            |                         |               |
|--|-----------------|--|--|---------|--|----------|--|-------------------|-------------------------------|------------|-------------------------|---------------|
|  | VolumelInstance |  |  | Running |  | t2.micro |  | 2/2 checks passed | <a href="#">View alarms +</a> | us-east-1b | ec2-34-229-206-11.co... | 34.229.206.11 |
|--|-----------------|--|--|---------|--|----------|--|-------------------|-------------------------------|------------|-------------------------|---------------|

## EBS Volume Creation and Attachment

A 10 GiB Magnetic (standard) EBS volume was created in the `us-east-1a` availability zone within the same region. The volume was created on December 14, 2025, at 11:32 GMT-6 and attached to the EC2 instance. On the instance, the volume appeared as device `/dev/xvdf`.

Volume settings

Volume type [Info](#)

Magnetic (standard) ▼

Size (GiB) [Info](#)

10 ▼

Min: 1 GiB, Max: 1024 GiB.

IOPS [Info](#)

Not applicable

Throughput (MiB/s) [Info](#)

Not applicable

Availability Zone [Info](#)

use1-az2 (us-east-1a) ▼

| Type     | Size          | IOPS | Throughput | Snapshot ID | Source Volume ID | Created                | Availability Zone     | Volume State |   |
|----------|---------------|------|------------|-------------|------------------|------------------------|-----------------------|--------------|---|
| standard | 10 GiB        | -    | -          | -           | -                | 2025/12/14 11:32 GMT-6 | use1-az2 (us-east-1a) | Available    | N |
| 20       | Delete volume |      |            |             |                  |                        |                       |              |   |
| 20       | Attach volume |      |            |             |                  |                        |                       |              |   |
| 20       | Detach volume |      |            |             |                  |                        |                       |              |   |
| 10 GiB   | -             | -    | -          | -           | -                | 2025/12/14 11:35 GMT-6 | use1-az4 (us-east-1b) | In-use       |   |

## Volume Formatting and Mounting

Root privileges were obtained (using `sudo su`), then block devices were listed with `lsblk` to confirm the appearance of `xvdf` (10G). The filesystem status was checked and revealed no existing filesystem. The volume was formatted to `ext4` and a mount point directory (`/application_data`) was created. The volume was mounted and verified via `df -h`, confirming 9.8G available and properly mounted on `/application_data`.

```
└─xvda128 259:1 0 10M 0 part /boot/efi
xvdf      202:80 0 10G 0 disk
└─┘
```

```
[ec2-user]# file -s /dev/xvdf
/dev/xvdf: data
```

```
ec2-user]# mkfs -t ext4 /dev/xvdf
021)
```

```
ec2-user]# mkdir /application_data
ec2-user]# |
```

```
point does not exist.
ec2-user]# mount /dev/xvdf /application_data
ec2-user]# |
```

```
[ec2-user]# df -h

```

| Filesystem   | Size | Used | Avail | Use% | Mounted on        |
|--------------|------|------|-------|------|-------------------|
| devtmpfs     | 4.0M | 0    | 4.0M  | 0%   | /dev              |
| tmpfs        | 481M | 0    | 481M  | 0%   | /dev/shm          |
| tmpfs        | 193M | 448K | 192M  | 1%   | /run              |
| /dev/xvda1   | 8.0G | 1.6G | 6.4G  | 20%  | /                 |
| tmpfs        | 481M | 0    | 481M  | 0%   | /tmp              |
| /dev/xvda128 | 10M  | 1.3M | 8.7M  | 13%  | /boot/efi         |
| tmpfs        | 97M  | 0    | 97M   | 0%   | /run/user/1000    |
| /dev/xvdf    | 9.8G | 24K  | 9.3G  | 1%   | /application_data |

## Data Creation and Testing

Test files were created within the mounted directory:

- testData.txt with sample content
- file2.txt containing “Additional data”
- timestamp.txt containing a timestamp

```
/dev/xvdf 9.8G 24K 9.3G 1% /application_data
[ec2-user]# echo "this is a sample file that goes in application_data directory" > /application_data/testData.txt
[ec2-user]# cat /application_data/testData.txt
this is a sample file that goes in application_data directory
[ec2-user]# |
```

The files' existence was verified with `ls -la /application_data/`, which also revealed the lost+found directory (standard for ext4 filesystems). File contents were confirmed with `cat` commands.

```
ec2-user]# ls -la /application_data/

4096 Dec 14 17:46 .
261 Dec 14 17:41 ..
16 Dec 14 17:46 file2.txt
16384 Dec 14 17:40 lost+found
62 Dec 14 17:44 testData.txt
29 Dec 14 17:46 timestamp.txt
```

## Volume Unmounting

After changing the working directory to avoid the mount point, the volume was unmounted using `umount /dev/xvdf`. The unmount was verified by checking the output of `df -h` to ensure `/dev/xvdf` no longer appeared in the list of mounted filesystems.

```
ec2-user]# umount /dev/xvdf
ec2-user]#
```

## Snapshot Creation

A snapshot of the detached 10GB volume was created with the following details:

- Description: "Snapshot of application\_data..."
- Size: 10 GiB
- Type: Standard

| Snapshot ID | Volume ID | Size   | Description                   | Type     |
|-------------|-----------|--------|-------------------------------|----------|
| 1           | -         | 10 GiB | Snapshot of application_da... | Standard |

The snapshot captured all test data files present on the volume.

## Volume Creation from Snapshot (Data Migration)

A new 15 GiB General Purpose SSD (gp3) volume was created from the snapshot, resulting in an upgrade from the original 10GB Magnetic volume. The new volume offered improved

performance with 3000 IOPS and 125 MiB/s throughput. It was created in the us-east-1b availability zone, matching the EC2 instance's location.

**Volume type** | [Info](#)

### General Purpose SSD (gp3)

Size (GiB) | Info

15

Min: 1 GiB, Max: 65536 GiB.

## IOPS | Info

3000

Min: 3000 IOPS, Max: 80000 IOPS.

Throughput (MiB/s) | Info

125

Min: 125 MiB, Max: 2000 MiB. Baseline: 125 MiB/s.

## Availability Zone | Info

use1-az4 (us-east-1b)

## Fast snapshot restore | Info

|   | NAME | TYPE | SIZE   | PROCESSES | MEMORY | STATUS          | START TIME | END TIME               |
|---|------|------|--------|-----------|--------|-----------------|------------|------------------------|
| ✓ | gp3  | gp3  | 15 GiB | 3000      | 125    | snap-0457858... | -          | 2025/12/14 11:53 GMT-6 |
|   |      |      |        |           |        |                 |            | us-east-1b             |

## Data Migration Verification

The new 15GB SSD volume was attached to the EC2 instance and identified as `/dev/xvdg`. A new mount point (`/newdata`) was created and the volume mounted there. File listings confirmed the presence of all original data files and the `lost+found` directory. The integrity of migrated data was verified by reading file contents. After verification, the volume was unmounted.

```

[ec2-user@ip-172-31-10-139 ~]$ sudo lsblk
[ec2-user@ip-172-31-10-139 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
xvda         202:0    0   8G  0 disk
├─xvda1      202:1    0   8G  0 part /
├─xvda127    259:0    0   1M  0 part
└─xvda128    259:1    0  10M  0 part /boot/efi
xvdg         202:96    0  15G  0 disk

```

```

c2-user]# mkdir /newdata
c2-user]# mount /dev/xvdg /newdata
c2-user]# ls -la /newdata/

4096 Dec 14 17:46 .
276 Dec 14 17:56 ..
16 Dec 14 17:46 file2.txt
16384 Dec 14 17:40 lost+found
62 Dec 14 17:44 testData.txt
29 Dec 14 17:46 timestamp.txt
c2-user]# |

```

```

[ec2-user@ip-172-31-10-139 ~]$ cd /home/ec2-user
[ec2-user@ip-172-31-10-139 ~]$ umount /dev/xvdg

```

## S3 Bucket Creation

An S3 bucket named my-volume-bucket-XXXXXXXXXX was created in the US East (N. Virginia) region. The bucket contained a pre-existing index.html file (273 bytes).

```

$ aws s3 ls --profile
2025-12-14 11:59:51 my-volume-bucket-58355t
|  Name
|
|  my-volume-bucket-XXXXXXXXXX

```

```
2025-12-14 12:02:17      273 index.html
aws s3 ls s3://my-volume-bucket- --profile
```

## AWS CLI Configuration and Testing

The AWS CLI was configured with a profile named 2025-12-14. Successful connection and access were verified by listing the bucket and its contents using CLI commands.

## File Upload Operations via CLI

Three local test files (test1.txt, test2.txt, test3.txt) were created and each uploaded to the S3 bucket via AWS CLI. File listings confirmed successful uploads. A deletion operation removed test1.txt from the bucket, and its absence was verified.

```
2025-12-14 12:02:17      273 index.html
$ echo "Test file 1" > test1.txt
echo "Test file 2" > test2.txt
echo "Test file 3" > test3.txt
$
```

```
upload: ./test3.txt to s3://my-volume-bucket-365557/test3.txt
$ aws s3 ls s3://my-volume-bucket- --profile
2025-12-14 12:02:17      273 index.html
2025-12-14 12:09:36       12 test1.txt
2025-12-14 12:10:01       12 test2.txt
2025-12-14 12:10:12       12 test3.txt
$
```

## Directory Sync to S3

A local directory (local-sync) containing a test file (sync1.txt) was created and its contents verified. The entire directory structure was uploaded to S3 using the `aws s3 sync` command.

```
$ mkdir local-sync
$ echo "Sync test" > local-sync/sync1.txt
$ cat local-sync/sync1.txt
Sync test

$ aws s3 sync local-sync/ s3://my-volume-bucket- /syncd --profile
upload: local-sync/sync1.txt to s3://my-volume-bucket- /syncd/sync1.txt
```

## S3 Bucket Versioning

Versioning was enabled on my-volume-bucket-XXXXXXXXXX. A test file (version-test.txt) was uploaded twice with different content, creating two versions. The existence of multiple object versions was confirmed using the AWS CLI.

### Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and recover from both unintended user actions and application failures. [Learn more](#)

#### Bucket Versioning

☐ Suspend

This suspends the creation of object versions for all operations but preserves any existing object versions.

☒ Enable

ⓘ After enabling Bucket Versioning, you might need to update your lifecycle rules to manage previous versions of objects.

#### Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions.  
Disabled

```
upload: ./version-test.txt to s3://my-volume-bucket-XXXXXXXXXX/version-test.txt
aws s3 cp version-test.txt s3://my-volume-bucket-XXXXXXXXXX/ --profile
```

## Cross-Region Replication Setup

A destination bucket (bucket-volume-XXXXXXXXXX) was created in the US West (Oregon) region, with versioning enabled. A replication rule named “Replicate To Oregon” was configured to apply to all objects in the source bucket. AWS automatically created the required IAM role to facilitate replication.

☐ [bucket-volume-XXXXXXXXXX](#) US West (Oregon) us-west-2

### Replication rule configuration

#### Replication rule name

Replicate To Oregon

Up to 255 characters. In order to be able to use CloudWatch metrics to monitor the progress of your replication rule, the replication rule name must

#### Status

Choose whether the rule will be enabled or disabled when created.

☒ Enabled

☐ Disabled

...

## Cross-Region Replication Testing

A new test file (new-test1.txt) was uploaded to the source bucket in N. Virginia. Replication was verified by confirming the file's appearance in the Oregon bucket on December 14, 2025, at 12:26:22. This demonstrated successful cross-region replication for new objects.

```
aws s3 cp s3://bucket-volume-58355t/new-test1.txt s3://my-volume-bucket-58355t/new-test1.txt --profile
upload: ./new-test1.txt to s3://my-volume-bucket-58355t/new-test1.txt
2025-12-14 12:26:22      27 new-test1.txt
```

**bucket-volume-58355t** info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (1)

Copy S3 URI

Copy URL

Down

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket.

Find objects by prefix

Show versions

|                          | Name                                                                                                              | Type | Last modified                           |
|--------------------------|-------------------------------------------------------------------------------------------------------------------|------|-----------------------------------------|
| <input type="checkbox"/> |  <a href="#">new-test1.txt</a> | txt  | December 14, 2025, 12:26:22 (UTC-06:00) |

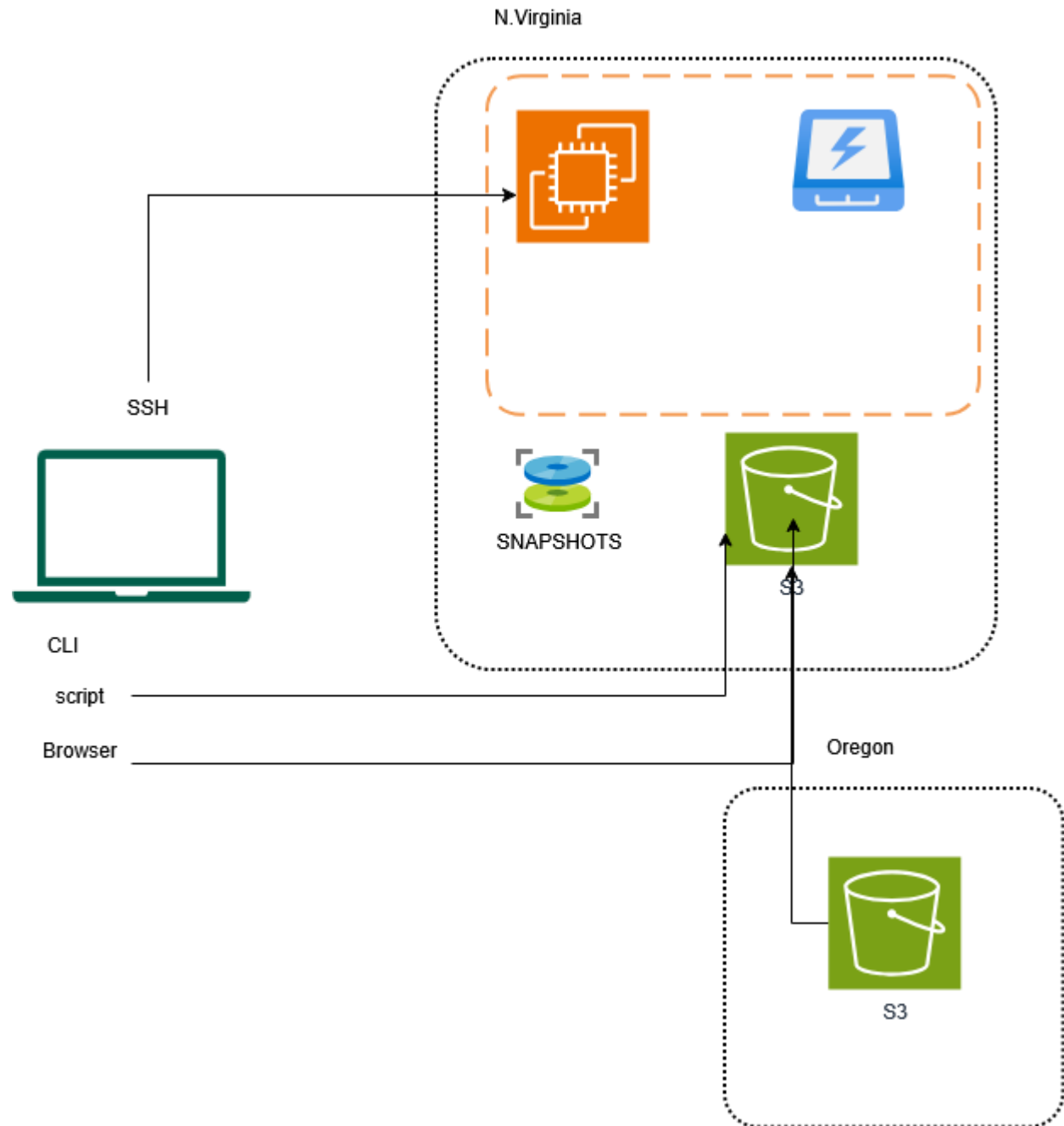
## Key Accomplishments Summary

- **EBS Volume Management:** Created, attached, formatted, mounted, and unmounted a 10GB Magnetic volume. Sample data was created to simulate production workloads.
- **Snapshot & Data Migration:** Successfully created a snapshot of the 10GB volume, restored it as a 15GB SSD, and verified complete data integrity after migration.
- **S3 Operations:** Established an S3 bucket, performed file uploads and deletions, and utilized directory synchronization.
- **Advanced S3 Features:** Enabled and tested bucket versioning, maintained multiple object versions, and performed advanced bucket operations.
- **Cross-Region Replication:** Set up and verified replication for new objects between the N. Virginia and Oregon regions.

## Skills Demonstrated

1. Linux System Administration – mounting, formatting, and file operations
2. AWS EC2 – instance management and SSH access
3. AWS EBS – volume creation, attachment, snapshotting, and restoration
4. Data Migration – volume-to-volume migration via snapshots
5. AWS CLI – configuration, S3 operations, and API calls
6. AWS S3 – bucket management, versioning, and replication
7. Disaster Recovery – cross-region data redundancy
8. Infrastructure Documentation – collection of 45 reference screenshots

## Architecture:



c

## References:

1. AWS Documentation: <https://docs.aws.amazon.com/>

**2. AWS CLI Reference:**

**<https://awscli.amazonaws.com/v2/documentation/api/latest/index.html>**

**3. AWS Free Tier: <https://aws.amazon.com/free/>**

**4. AWS Architecture Center: <https://aws.amazon.com/architecture/>**

**5. AWS Well-Architected Framework:**

**<https://aws.amazon.com/architecture/well-architected/>**